

En esta actividad no deberán crear el programa desde cero, sino que se les entrega un código fuente ya desarrollado, donde deberán:

1. Crear el archivo. **Ejemplo: nombre_archivo.c**
2. Compilar el programa. **gcc nombre_archivo.c -o nombre_archivo**
3. Ejecutarlo. **./nombre_archivo**
4. Identificar y analizar los resultados de los procesos padre e hijos.
5. Observar y comparar con resultados que propusieron ustedes con la salida obtenida.

Ejercicio 1	
<pre>#include <stdio.h> #include <unistd.h> int main() { fork(); printf("Hola desde un proceso\n"); return 0; }</pre>	Preguntas: 1. ¿Cuántas veces se imprime el mensaje? 2. ¿Por qué se imprime más de una vez? 3. ¿Cuántos procesos existen después del fork()?
Ejercicio 2	
<pre>#include <stdio.h> #include <unistd.h> int main() { int pid; pid = fork(); if (pid == 0) { printf("Soy el hijo\n"); printf("PID del hijo: %d\n", getpid()); printf("PID de mi padre: %d\n", getppid()); } else { printf("Soy el padre\n"); printf("PID del padre: %d\n", getpid()); printf("PID del hijo creado: %d\n", pid); } return 0; }</pre>	Preguntas: 1. ¿Qué proceso imprime "Soy el hijo"? 2. ¿Qué proceso imprime "Soy el padre"? 3. ¿Qué valor devuelve fork() en el hijo? 4. ¿Qué valor devuelve fork() en el padre?
Ejercicio 3	
<pre>#include <stdio.h> #include <unistd.h></pre>	Preguntas:

```
int main() {
    fork();
    fork();

    printf("Final del programa\n");

    return 0;
}
```

1. ¿Cuántas veces se imprime "Final del programa"?
2. ¿Cuántos procesos se generan en total?
3. ¿Por qué dos fork() generan más de dos procesos?

Ejercicio 4

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int numero = 10;
    int pid;

    pid = fork();

    if (pid == 0) {
        numero = 20;
        printf("Hijo: numero = %d\n",
numero);
    } else {
        printf("Padre: numero = %d\n",
numero);
    }

    return 0;
}
```

Preguntas:

1. ¿Qué valor imprime el padre?
2. ¿Qué valor imprime el hijo?
3. ¿Por qué el cambio del hijo no afecta al padre?
4. ¿Los procesos comparten la misma memoria?

Ejercicio 5

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int pid;

    pid = fork();

    if (pid == 0) {
        printf("El hijo va a ejecutar
ls\n");
        execl("/bin/ls", "ls", NULL);

        printf("Este mensaje solo
aparece si falla execl\n");
    } else {
        printf("El padre continúa su
ejecución\n");
    }
}
```

Preguntas:

1. ¿Qué proceso ejecuta el comando ls?
2. ¿Qué hace execl()?
3. ¿Aparece el mensaje "Este mensaje solo aparece si falla execl"?
4. ¿Por qué?

<pre> return 0; }</pre>	
Ejercicio 6	
<pre>#include <stdio.h> #include <unistd.h> #include <sys/wait.h> int main() { int pid; pid = fork(); if (pid == 0) { printf("Hijo iniciado\n"); printf("PID hijo: %d\n", getpid()); execl("/bin/date", "date", NULL); printf("Error al ejecutar date\n"); } else { printf("Padre iniciado\n"); printf("PID padre: %d\n", getpid()); printf("Esperando al hijo...\n"); wait(NULL); printf("El hijo terminó. Padre finaliza.\n"); } return 0; }</pre>	Preguntas: 1. ¿Qué comando ejecuta el hijo? 2. ¿Qué hace wait(NULL)? 3. ¿Por qué el padre imprime el mensaje final después del hijo? 4. ¿Qué pasaría si sacamos wait(NULL)?
Ejercicio 7	
<pre>#include <stdio.h> #include <unistd.h> #include <sys/wait.h> int main() { int pid1, pid2; pid1 = fork(); if (pid1 == 0) { printf("Hijo 1 ejecuta uptime\n"); execl("/usr/bin/uptime",</pre>	Preguntas: 1. ¿Cuántos hijos crea el padre? 2. ¿Qué comando ejecuta cada hijo? 3. ¿Por qué hay dos wait(NULL)? 4. ¿El orden de salida siempre es igual? 5. ¿Por qué puede cambiar?

```

"uptime", NULL);
    return 1;
}

pid2 = fork();

if (pid2 == 0) {
    printf("Hijo 2 ejecuta df -
h\n");
    execl("/bin/df", "df", "-h",
NULL);
    return 1;
}

printf("Padre creó dos hijos\n");
printf("PID hijo 1: %d\n", pid1);
printf("PID hijo 2: %d\n", pid2);

wait(NULL);
wait(NULL);

printf("Ambos hijos
terminaron\n");

return 0;
}

```

Ejercicio 8

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pid;
    int contador = 1;

    pid = fork();

    if (pid == 0) {
        contador++;
        printf("Hijo 1 - contador:
%d\n", contador);

        fork();

        printf("Proceso generado por
el hijo - PID: %d\n", getpid());
    } else {
        contador = contador + 5;
        printf("Padre - contador:
%d\n", contador);

        wait(NULL);
    }
}

```

Preguntas:

1. ¿Cuántos procesos se generan en total?
2. ¿Qué valor de contador imprime el padre?
3. ¿Qué valor de contador imprime el hijo?
4. ¿El segundo fork() lo ejecuta el padre o el hijo?
5. ¿Cuántas veces aparece "Proceso generado por el hijo"?
6. ¿Por qué el valor de contador no es igual en todos los procesos?

```
        printf("Padre finaliza\n");  
    }  
  
    return 0;  
}
```